

DIP Exam Master Guide

2026-09-20 · @Someone

Read this first

Everything in your 34 note pages is covered below, in the order that builds understanding, with each formula written out and each worked example re-computed. Where your notes look wrong or unclear, I flag it under **Check** so you don't memorise a mistake.

Order to study tonight (highest exam value first):

1. Part 3: contrast, negatives, stretching, histogram equalization table (numerical questions are near-certain).
2. Part 4: convolution, Prewitt/Sobel/Laplacian/LoG numbers, Canny steps.
3. Part 2: interpolation (nearest neighbour and bilinear worked examples, padding).
4. Part 5: HSV conversion and false colour composites.
5. Part 6: circularity, Moore tracing, Hough, MLP features.
6. Part 1: definitions and modalities (theory answers).

Exam habit: for every numerical, write the formula, substitute, then state the result in one line. For every theory question, give definition, one formula or example, one application.

Check notes marked this way are places where your notes disagree with the standard textbook (Gonzalez and Woods). If your professor taught it differently, write what he taught, but know the standard form.

Part 1: Introduction, pipeline, imaging modalities, sensors

1.1 What is Digital Image Processing (DIP)?

A digital image is a continuous light signal (from the light spectrum) converted into a **pixelated matrix**. DIP means processing those matrices with algorithms, mainly image in, image out (enhance, filter, restore, compress).

Acquisition sources and applications: face recognition, Google Maps, UAV (drone) photogrammetry, Instagram filters, MRI scans, satellite imagery, CCTV cameras. Satellite and drone imaging come under **Earth Observation Systems**.

DIP vs Computer Vision (CV): DIP transforms an image into a better image. CV extracts meaning from an image (recognise, detect, decide).

Example from your notes (selfie unlock):

Step	Belongs to
Original selfie	Input
Increase brightness	DIP
Remove noise	DIP
Detect face	DIP/CV boundary (your notes say "detect noise": Check , it should be detect face)
Recognise person	Computer Vision
Unlock phone	Computer Vision

- Google Maps uses **image stitching**; drone mapping uses **orthomosaic generation** (many overlapping photos corrected into one geometrically accurate map).
- **Feature map:** output generated from image information by neural processing at multiple layers.
- **Major recent advances:** YOLO versions, CNNs, AlexNet, ResNet, multimodal AI, Vision Transformers.
- **YCbCr:** Y is luma (brightness), Cb and Cr are colour-difference channels; used in digital image and video elements (JPEG, video codecs).
- **X-ray tip:** bones appear white; apply inverse (negative) transformations to visualise them better against muscles and soft spots.

1.2 Sampling grid and ground units (your first page)

A sensor cell (e.g. 2 mm × 2 mm unit) covers a ground patch that grows with distance: at 10 m it covers about 2 ft × 2 ft, and at a much larger distance (100 m) a much larger patch.

The point: **pixel ground size depends on sensor distance, sampling rate and quantization levels. Check:** distance grew 10×, so the patch should grow 10× (about 20 ft) if scaling is linear; verify the 200 ft figure against the slide before writing it.

1.3 DIP pipeline (image formation)

Illumination $\rightarrow$ Reflection $f(x, y, \lambda) \rightarrow$ Optical lens $O(\cdot) \rightarrow$ Sensor plane $f(x, y) \rightarrow$ Sampling $f(i, j) \rightarrow$ Quantization $q(i, j) \rightarrow$ Digital image $g(i, j) \rightarrow$ Encoding / Storage / Transmission.

$$f(x, y) = i(x, y) \cdot r(x, y), \quad 0 < i(x, y) < \infty, \quad 0 < r(x, y) < 1$$

(i = illumination, r = reflectance; standard Gonzalez image formation model, add it as a bonus line.)

1.4 Early digital images: the perforated-strip system

Early digital pictures were transmitted using a **perforated paper strip**, and a grid of box units is drawn where boxes inside the person's boundary are kept blank. Your notes attribute this to "Sir Frank Macfarlane Burnet (Nobel 1960, Medicine)". **Check:** Burnet was an immunologist; the image system is the **Bartlane cable picture transmission system (1920s), by Harry Bartholomew and Maynard McFarlane**. Confirm with your slide, but do not write Burnet for an imaging question.

Moon images were received at Earth via radio and microwaves (USB = Unified S-band radiowave system), and they are **grayscale**, not RGB.

1.5 Imaging modalities (EM spectrum, see slide)

Modality	How the image forms	Key points from your notes
X-ray	Grayscale image; dense materials absorb more X-rays and appear white, less dense let more pass and appear darker	Bones white. Modern X-ray: high-pass filtering, Digital Radiography (DR)
CT (Computer Tomography)	Cross-section images reconstructed from many X-ray projections	Helical-type sensor approach (see slide)
PET (Positron Emission Tomography)	Tracer accumulates in metabolically active tissue instead of bones; gamma rays are detected	Result is bright spots at active regions

Modality	How the image forms	Key points from your notes
Infrared	Usually grayscale or false colour; wavelength about 0.75 μm to 14 μm (near IR to thermal IR)	Represents temperature , not visible colour. Uses: fever screening, fire detection, search and rescue
Multispectral	Typically 4 to 13 spectral bands, e.g. Blue, Green, Red, NIR, SWIR	NIR = Near IR, SWIR = Shortwave IR. Uses: water resources, land-use mapping, forest monitoring, agriculture, disaster management
Hyperspectral	100 to 300+ narrow spectral bands; measures a detailed spectral signature	Uses: crop disease detection, mineral exploration and mapping, cancerous cell detection
Microscopy	General microscopy is reflective in nature	Light microscopy detects crystalline structures such as deposits of stones and cholesterol

Destructive vs non-destructive testing on images: imaging methods such as X-ray, IR and ultrasound examine an object without damaging it (non-destructive); cutting a sample to look inside is destructive.

1.6 Components of a general-purpose image processing system

Problem domain → **Image sensors** → **Specialised image processing hardware** → **Computer**. The computer connects to: mass storage, image displays, hard copy devices, image processing software, and a network.

1.7 Human eye

- **Scotopic vision:** dim light, done by **rods** (many, no colour, high sensitivity).
- **Photopic vision:** bright light, done by **cones** (fewer, colour vision, concentrated at the fovea).
- **Glare limit:** upper limit of brightness the eye tolerates; subjective brightness follows a **brightness order** (relative, logarithmic-like sensitivity).
- Rods vs cones: rods for night, low detail, no colour; cones for day, fine detail, colour.

- Colour sensitivity used for luminance: green about 59%, red about 29%, blue about 11% (Part 5).

1.8 Image sensing and sensors

A sensor has: incoming **energy**, a **filter**, **sensing material**, **housing**, **power in**, and produces a **voltage waveform out**. Arrangements: single sensor, **line-based**, **array/matrix-based**.

CCD	CMOS
Charge is shifted across the chip and read at one output	Each pixel has its own amplifier and readout
Low noise, high quality	Cheaper, faster, low power, somewhat noisier
More power, more costly	Dominant in phone cameras

1.9 Resolution, quantization, file types, zooming

- **Resolution:** ability to distinguish closely spaced objects. 20 MP > 10 MP for the same field.
- **Spatial resolution:** the smallest detail an imaging system can clearly show.
- **Radiometric resolution:** ability to measure and record fine differences in light energy, expressed as **bit depth** (bits).
- **Image quantization:** reducing the number of colours or grayscale levels. It **groups** a continuous range of light intensities into one discrete code value. **Compression benefit:** fewer bits per pixel reduce file size. It is a **lossy process:** discarded information can never be perfectly recovered.

$$L = 2^{\{b\}}, \quad \text{Storage (bits)} = M \times N \times b$$

- **File types:** JPEG/JPG (lossy, photos), PNG (lossless, transparency), SVG (vector), TIFF (lossless, high quality, scientific/print).
- **Related terms:** image denoising, image saturation (clipping at max intensity), over-exposure (too bright, highlights lost), under-exposure (too dark, shadows lost).
- **Zooming:** **optical zoom** changes the lens and captures real detail; **digital zoom** enlarges pixels by interpolation and adds no new detail.

- **AI camera features** and the one table on the slide (compare with your slide copy).

Part 2: Image interpolation, resizing, padding, pulling

Image interpolation: a technique to estimate missing pixel values and calculate new pixel grids so an image scales up or down smoothly in a continuous manner. It is the engine behind **image resizing (upscaling / downscaling)**. Enlarging a 2×2 image to 4×4 makes the **image size** bigger while each **individual pixel size** stays the same.

Base example used throughout:

```
10, 20
30, 40
```

2.1 Nearest Neighbour (NN) interpolation

For every new pixel position, find the closest corresponding old pixel and **copy its exact value** (no averaging). No new or intermediate values are created. Enlarging causes harsh visible steps or blocks (**blocky / staircase effect, jagged edges**) because pixels are simply duplicated.

$$i_{\text{old}} = i_{\text{new}} \left[\left\lceil \left(\frac{r}{sf}, \frac{c}{sf} \right) \right\rceil \right]$$

r = row, c = column of the new pixel (positions like a matrix, starting from 1), sf = scaling factor (here $sf = 2$).

Result of 2×2 to 4×4 by NN (each old pixel becomes a 2×2 block):

```
10, 10, 20, 20
10, 10, 20, 20
30, 30, 40, 40
30, 30, 40, 40
```

Worked mappings (round halves up):

New cell	$r/sf, c/sf$	Rounded	Old cell	Value
(2, 2)	(1, 1)	(1, 1)	(1, 1)	10
(3, 3)	(1.5, 1.5)	(2, 2)	(2, 2)	40

New cell	r/sf, c/sf	Rounded	Old cell	Value
(4, 4)	(2, 2)	(2, 2)	(2, 2)	40

Check: your notes say new cell (2,2) maps to old (1,1) and then say the value is 40. Old (1,1) is **10**. The value 40 belongs to new cell (3,3), which you also work out ($\text{round}(3/2, 3/2) = (2,2) = 40$). Write it as in the table above.

2.2 Bilinear interpolation

Calculates a new pixel value as a **weighted average of the 2×2 grid of the four closest pixels**. It does linear interpolation **first horizontally, then vertically**. Weights depend on the distance of the new pixel from each neighbour. It **prevents the blocky staircase effect** of NN and produces smoother results.

With neighbours P11, P12 (top row) and P21, P22 (bottom row), and fractional offsets x (down) and y (across), $0 \leq x \leq 1$ and $0 \leq y \leq 1$:

$$P_{\text{new}} = P_{11}(1-x)(1-y) + P_{12}(1-x)y + P_{21}x(1-y) + P_{22}xy$$

Case $x = 0, y = 0.5$ (midpoint of top row): terms with x vanish, so

$$P_{\text{new}} = 0.5P_{11} + 0.5P_{12} = \frac{P_{11} + P_{12}}{2} = \frac{10 + 20}{2} = 15$$

Centre of the 2×2 ($x = y = 0.5$): average of all four = $(10 + 20 + 30 + 40)/4 = 25$.

2×2 to 3×3 by bilinear:

```
10, 15, 20
20, 25, 30
30, 35, 40
```

- **2×2 to 6×6 by bilinear** is a two-step process using the same P_{new} equation: (2×2) to (3×3) to (6×6).
- **Compression** is also possible: 2×2 to one pixel gives 25 (average of the four). Pulling then flattening is the same idea in reverse.
- **Why "linear"?** Interpolation is linear along each direction. In the equal-neighbour case above all weights are equal, which is why it reduces to a plain average. **Check:** in general the weights are not equal; they depend on distance (x, y).

2.3 Bicubic interpolation

Calculates a blended value from the **16 surrounding pixels (4×4 grid)** and applies a **non-linear mathematical curve (cubic spline)** across rows and columns. A new blended colour value is generated. Result: gentler transitions, sharper details and fewer artifacts than bilinear (but slower).

Check: your notes for 13/08 call this an "inverse distance-based weighting (IDW) approach". IDW is a different method. If your professor said it, write it as a remark; the standard description is a cubic convolution / cubic spline over a 4×4 neighbourhood.

On splines: a **closed path** returns to where it started (circular loop, continuous); an **open path** is one-ended.

Lanczos interpolation (partly written in your notes) uses a windowed sinc kernel, with a = window size:

$$L(x) = \begin{cases} \operatorname{sinc}(x) \cdot \operatorname{sinc}(x/a), & |x| < a \\ 0, & \text{otherwise} \end{cases}, \quad \operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

Method	Neighbours used	Quality	Speed
Nearest neighbour	1	Blocky, jagged	Fastest
Bilinear	4 (2×2)	Smooth, slightly soft	Medium
Bicubic	16 (4×4)	Smoothest, sharper detail	Slowest of the three

2.4 Pulling (inverse / backward mapping)

The algorithm **loops through every pixel on the new target canvas, maps its coordinates back to the original source image, and extracts the correct colour data.** This avoids holes that forward mapping would leave.

2.5 Image padding

Artificially expands the image by adding extra pixels around the borders. Needed so a kernel can be applied at edges and so output size is kept.

Type	What is added
Zero padding (constant)	Zeros (or another constant)
Replication (clamp)	Copies of the nearest edge pixel
Reflection (mirror)	Mirror image of pixels near the edge
Wrap (periodic)	Pixels from the opposite side

2.6 Band ratioing and distances

- **Band ratioing:** enhances specific surface features by dividing the digital number (DN) of one spectral band by another, pixel by pixel. Standard example, vegetation index:

$$\mathrm{NDVI} = \frac{\text{NIR} - \text{Red}}{\text{NIR} + \text{Red}}$$

- **Distance measures** between pixels (x_1, y_1) and (x_2, y_2) :

$$D_E = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}, \quad \text{quad } D_4 = |x_1 - x_2| + |y_1 - y_2| \quad (\text{Manhattan}), \quad \text{quad } D_8 = \max(|x_1 - x_2|, |y_1 - y_2|) \quad (\text{chessboard})$$

Part 3: Intensity transformations, contrast, histograms

3.1 Definitions to write word for word

- **Contrast:** variations in the intensity of pixels.
- **Tones:** variations in brightness, contrast and colour intensity across an image. Three tonal regions: **Highlights** (brightest areas), **Midtones** (mid-range of tones), **Shadows** (darkest areas).
- **Negative:** inverts the intensity levels of the original image.
- **Diapositive:** a transparent positive photograph that displays colours and tones exactly as they appear in reality. Its transformation is the identity.

$$\begin{aligned} \text{Negative: } s &= 255 - r \quad (\text{8-bit, } r \in [0, 255]) \quad \text{quad} \\ \text{General: } s &= (L-1) - r \end{aligned}$$

$$\text{Diapositive (identity): } s = r$$

r = input pixel, s = output pixel.

3.2 Reading a histogram

Histogram: x-axis = intensity (DN 0 to 255), y-axis = frequency (number of pixels).

- Mass near 0: dark image. Mass near 255: bright image.
- **Narrow** histogram: low contrast. **Wide** spread: high contrast.
- **Higher variance of the intensity distribution means higher contrast** (variance is highly correlated with contrast).

Question 1 (your notes): Image A has its peak at intensity 30, Image B has its peak at 190. Which is brighter? **Answer: Image B**, because more pixels are concentrated towards the brighter (255) side, with the maximum peak at 190.

Question 2 (your notes): Three histograms A, B, C; C is a broad, flat spread from 0 to 255. **Answer: Image C has the highest variance, hence the highest contrast.**

3.3 Contrast shifting vs stretching

Contrast shifting: adjusts pixel intensities to change how bright and dark areas look relative to each other. Adding a constant moves every value equally:

```
1,1,1,1
2,2,2,2
3,3,3,3
4,4,4,4
```

Adding 5 to each pixel gives rows of 6, 7, 8, 9. **The contrast is the same (differences unchanged) but shifted; intensity (brightness) is increased.**

Contrast stretching: expands the range of pixel intensity values so dark pixels get darker and bright pixels get brighter.

$$B(x,y) = \frac{I_B(x,y) - I_{\{B,\min\}}}{I_{\{B,\max\}} - I_{\{B,\min\}}} \times 255$$

Check: your page writes " $\times 2.55$ "; the multiplier for an 8-bit image is 255.

Example: an image with min 50 and max 150: a pixel of 100 becomes $(100 - 50)/(150 - 50) \times 255 = 127.5$, about 128.

Binary thresholding: converts a grayscale image to a black-and-white binary image by comparing each pixel intensity to a set cutoff value T : output = 255 (or 1) if pixel $> T$, otherwise 0.

Level slicing: highlights a specific range of pixel intensities while dimming or ignoring the rest.

3.4 Transformation family (contrast stretching functions, 03/08)

Type	Function	Effect
Linear	$g = \alpha f + \beta$	α controls contrast (gain), β controls brightness (bias)
Logarithmic	$g = b \log(\alpha f + 1)$	Expands dark values, compresses bright values
Exponential	$g = b(e^{\alpha f} - 1)$	Opposite of log: expands bright values, compresses dark
Power law, $k > 1$	$g = \alpha f^k$	Darkens; expands bright range
Power law, $0 < k < 1$	$g = \alpha f^k$	Brightens; expands dark range

Textbook form of the power law is $s = c \cdot r^\gamma$ (gamma correction), with c a scaling constant.

3.5 Colour contrast and luminance (02/09)

Euclidean colour distance between two colours (R_1, G_1, B_1) and (R_2, G_2, B_2) :

$$\text{Contrast} = \sqrt{(R_1 - R_2)^2 + (G_1 - G_2)^2 + (B_1 - B_2)^2}$$

White (255,255,255) against black (0,0,0) gives $\sqrt{3 \times 255^2} \approx 441.7$.

Human-eye sensitivity weights: **Green 59%, Red 29%, Blue 11%.**

Your worked example (pixel $R = 255$, $G = 255$, $B = 0$, i.e. yellow):

$$\text{contrast} = \frac{255 \times 0.59 + 255 \times 0.29 + 0}{3} = \frac{224.4}{3} = 74.8 \approx 75$$

Check: the standard luminance is $Y = 0.29R + 0.59G + 0.11B$, which for this pixel is **224.4** with no division by 3. Use the $\div 3$ version only if your professor's method includes it (your notes show it).

3.6 Histogram equalization (your 42-pixel table, 3-bit image, $L = 8$)

Goal: spread the intensity histogram across the full range to increase overall contrast.

$$s_k = \operatorname{round}\left(\frac{(L-1) \cdot \operatorname{CDF}(r_k)}{\sum_{j=0}^{L-1} n_j}\right), \quad \forall k$$

Steps: (1) count frequency of each level, (2) compute cumulative sum, (3) divide by total pixels for CDF, (4) multiply by $L - 1 = 7$, (5) round to get new level.

Original brightness	Frequency	CDF	CDF $\times 7$	Transformed (rounded)
0	0	0	0	0
1	4	4/42	0.67	1
2	5	9/42	1.5	2
3	9	18/42	3.0	3
4	9	27/42	4.5	5
5	12	39/42	6.5	7
6	3	42/42	7	7
7	0	42/42	7	7

Half values round up (1.5 to 2, 4.5 to 5, 6.5 to 7), as in your notes.

Check: your frequencies for levels 3 and 4 read 8 and 10, but your CDF numerators (18/42, 27/42) require 9 and 9 ($4 + 5 + 9 = 18$; $18 + 9 = 27$). The counts must sum to 42, so use 9 and 9.

Result: brightness values are enhanced (overall contrast stretching). Levels 5, 6, 7 all merge into 7, so output level 7 now has $12 + 3 + 0 = 15$ pixels; equalization cannot make a perfectly flat histogram in a discrete image. Your note "7 has 3 occurrences" refers to the original level 6, which has 3 pixels.

New histogram: level 1: 4, level 2: 5, level 3: 9, level 5: 9, level 7: 15 (total 42).

Part 4: Spatial filtering, kernels, edge detection

4.1 Convolution and kernels

A **filter** (used for removal of noise, smoothing, sharpening, edge detection) works by **convolution**: a small matrix called a **kernel** (mask) slides over the image, and each output pixel is a weighted sum of the neighbourhood.

$$g(x, y) = \sum_{i=-m}^{+m} \sum_{j=-n}^{+n} K(i, j) \cdot f(x-i, y-j)$$

Here x runs 0 to r (rows), y runs 0 to c (columns), (x, y) are image coordinates, and K is the kernel of size $(2m+1) \times (2n+1)$. For a 3×3 kernel, $m = n = 1$, so i and j go from -1 to $+1$ (the coordinate grid in your notes labels rows $-1r, 0r, +1r$ and columns $-1c, 0c, +1c$).

Worked example (mean filter): a 3×3 window where every pixel is 30 except the centre, which is 0. Kernel = all ones, divided by 9.

$$\frac{30 \cdot 1 \cdot 8 + 0 \cdot 1}{9} = \frac{240}{9} = 26.67$$

The centre pixel changes from 0 to about 26.6667, so the noise spike is smoothed out.

Frequency idea: the lower the relative contrast between neighbouring pixels, the lower the frequency; sharp change means high frequency (edges, noise).

Filter type	Promotes	Examples	Kernel sum
Low-pass (LPF)	Low-frequency areas (smooth regions), removes noise	Mean, Gaussian. Anything "averaging" is a low-pass filter	1 (after dividing)
High-pass (HPF)	High-frequency areas (edges, detail)	Prewitt, Sobel, Laplacian, LoG	0 (all high-pass filters are zero-sum)

Boundary vs edge: all boundaries are edges, but not all edges are boundaries (an edge is any intensity discontinuity; a boundary is a closed outline of an object). Edge detection uses contrast and **geometric context** ("low frequency imaging" is the opposite: smooth regions).

4.2 Prewitt kernel

3×3 digital filter to detect horizontal and vertical edges.

Vertical-edge kernel (detects intensity change left to right):

```
-1, 0, 1
-1, 0, 1
-1, 0, 1
```

Horizontal-edge kernel (detects change top to bottom):

```
-1, -1, -1
0, 0, 0
1, 1, 1
```

Worked example (horizontal edge, top rows 200, bottom rows 50): response = $-(200 \times 3) + (50 \times 3) = -600 + 150 = -450$ at the edge row (flat regions give 0). Your notes show -450 -450 in the output.

4.3 Sobel kernel

Small 3×3 matrix for edge detection by calculating the **approximate image gradient**. It is Prewitt with extra weight (2) on the centre row/column, which also smooths slightly.

```
-1, 0, 1
-2, 0, 2
-1, 0, 1
```

This detects **vertical** edges. Transpose it for horizontal edges:

```
-1, -2, -1
0, 0, 0
1, 2, 1
```

Worked example (top 200, bottom 50): $-(200 \times 4) + (50 \times 4) = -800 + 200 = -600$ (your notes: -600 -600). Rule from your notes: **Sobel to identify a horizontal edge detects change in the vertical direction.**

Diagonal edge kernels (from your notes):

```
0, -1, -1
1, 0, -1
1, 1, 0
```

```
1, 1, 0
1, 0, -1
0, -1, -1
```

4.4 Gaussian kernel (smoothing)

3×3 approximation, with the weights summing to 16 (so divide by 16 to keep brightness):

```
\frac{1}{16}\begin{bmatrix}1&2&1\\2&4&2\\1&2&1\end{bmatrix}
```

It is a **weighted average**, giving more weight to the centre. It smooths and removes noise, so it is a low-pass filter.

```
G(x)=\frac{1}{\sqrt{2\pi}\sigma}e^{-\frac{(x-\mu)^2}{2\sigma^2}}
```

Check: your page shows the exponent as $-\frac{1}{2}(x - \mu)^2$ without the σ^2 ; the full form divides by σ^2 .

4.5 Laplacian filter

Second-derivative filter; detects edges in **both horizontal and vertical** directions (isotropic).

```
0, -1, 0
-1, 4, -1
0, -1, 0
```

The 8-neighbour version (diagonals included):

```
-1, -1, -1
-1, 8, -1
-1, -1, -1
```

Check: on your page (after "Gaussian then Laplacian") the centre of this matrix reads 0, but a zero-sum high-pass kernel with eight -1 neighbours needs a centre of **8**. Zero centre would not sum to zero.

Sharpening with the Laplacian (syllabus): if the centre is -4 use $g = f - \nabla^2 f$, if the centre is $+4$ use $g = f + \nabla^2 f$. **High-boost filtering:** $g = f + k(f - \text{blurred } f)$, with $k > 1$.

4.6 Laplacian of Gaussian (LoG)

Apply Gaussian smoothing first, then the Laplacian: it **enhances edges and denoises**.

5×5 LoG kernel from your notes:

```
0,0,-1,0,0
0,-1,-2,-1,0
-1,-2,16,-2,-1
0,-1,-2,-1,0
0,0,-1,0,0
```

Sum of all entries = 0 (check: $-1 - 4 + 10 - 4 - 1 = 0$), so it is a **high-pass filter**. Contrast: the simple Gaussian was a low-pass filter (weights positive, sum divided to 1). **Check:** your note says the reason is "divided by 16"; the real reason is that LoG is zero-sum with a negative surround while Gaussian is all-positive.

4.7 Edge strength, direction, and the Canny detector

```
\text{Edge strength} = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}, \quad \text{Edge direction } \theta = \tan^{-1}\left(\frac{dy}{dx}\right)
```

(dy = vertical gradient, dx = horizontal gradient; use `atan2` in code so the quadrant is right.)

Canny edge detector steps:

1. Smooth with a Gaussian filter (reduce noise).
2. Compute gradient strength and direction (Sobel).
3. **Non-max suppression:** thin the edges by keeping only local maxima along the gradient direction.
4. **Double threshold and hysteresis:** connect edges.

Non-max suppression example (your notes):

```
270,40,85
119,450,60
90,220,100
```

Only the largest value (450) survives; every other cell becomes 0, leaving a one-pixel-thin edge:

```
0, 0, 0
0, 450, 0
0, 0, 0
```

Hysteresis (your page says "concept of hypothesis": **Check**, the word is *hysteresis*): use two thresholds, high and low.

- Gradient above the high threshold: **strong edge**, keep.
- Below the low threshold: discard.
- In between: keep **only if connected to a strong edge**.

This stops a lone noisy value (the odd 384 among 38, 39, 40 in your sketch, marked "may be a noise!!") from being kept as an edge, while continuous edges with slightly weak segments (39, 40, 41) stay connected.

4.8 Convolution inside a CNN; padding and stride

- A **kernel** in a CNN works like the filters above, but its weights are learned.
- A 3×3 kernel on a 256×256 image gives a **254×254** output (edge pixels are lost).
- **Padding** keeps size: add a border of rows and columns, filled with 0s (zero padding), 1s, or replicated values.
- **Stride** = step size of the window: stride 1 moves one pixel, stride 2 moves two pixels (**compresses** the output).

$$\text{Output size} = \left\lfloor \frac{N + 2p - k}{s} \right\rfloor + 1$$

N = input size, k = kernel size, p = padding, s = stride. Check: $(256 + 0 - 3)/1 + 1 = 254$.
 "Same" output with stride 1 needs $p = (k - 1)/2$, so $p = 1$ for a 3×3 kernel.

Self-attention (Vision Transformers): unlike a CNN, which only sees a local neighbourhood, self-attention lets one region relate to any other region in the image, creating **global context** (your sketch of the person with arrows between distant corners).

Augmentation (to enlarge training data): rotation, flip, stretching, adding noise.

Noise removal by type (syllabus): Gaussian noise: Gaussian/mean filter. Salt-and-pepper noise: **median filter** (takes the middle value of the sorted window, keeps edges).

Note: **segmentation is a useful technique in image classification**.

Part 5: Colour: false colour composites and HSV

5.1 False Colour Composite (FCC)

Definition: a colour image built by assigning spectral bands (including wavelengths **beyond the visible spectrum**, e.g. near infrared) to the display's Red, Green and Blue channels. Each band is a grayscale image; think of a red torch, a green torch and a blue torch each lighting one of the R, G, B image planes (your sketch).

Counting composites for 3 bands:

```
\#\text{ composites} = 3! = 6, \quad \#\text{ true colour} = 1, \quad \quad
\#\text{ false colour} = 3! - 1 = 5
```

- **True colour composite:** bands placed as they are seen (Red band to R, Green band to G, Blue band to B).
- Standard remote-sensing FCC: NIR to R, Red to G, Green to B, so healthy vegetation appears bright red. Reason: **IR reflects strongly on vegetation and plants.**

Why HSV is used: if you histogram-equalize the R, G and B channels separately and then recombine them, the colour image is **distorted** (colours shift, since the balance between channels changes). So the **HSV approach** is adopted: convert to HSV, equalize only the **V** (brightness) channel, keep H and S, convert back to RGB.

5.2 HSV model (Hue, Saturation, Value)

- **Hue (H):** the pure colour, an angle on the colour wheel (0° to 360°).
- **Saturation (S):** range 0 to 1. 0 = impure (grayish / washed out), 1 = 100% pure colour.
- **Value (V):** brightness. Your dot analogy: a 0.25 mm dot growing to a 1 mm dot means value is increased.

Hue wheel (angles): Red 0° (also 360°), Yellow 60°, Green 120°, Cyan 180°, Blue 240°, Magenta 300°.

Check: your wheel labels yellow as 90°. In the standard HSV wheel the six primaries and secondaries are 60° apart, so yellow is **60°** (your other labels 120, 180, 240, 300 already follow the 60° spacing).

5.3 RGB to HSV: the algorithm (write in this order)

Step 1, normalise:

$$R' = \frac{R}{255}, \quad G' = \frac{G}{255}, \quad B' = \frac{B}{255}$$

Step 2, extremes and range:

$$C_{\max} = \max(R', G', B'), \quad C_{\min} = \min(R', G', B'), \quad \Delta = C_{\max} - C_{\min}$$

Step 3, value:

$$V = C_{\max}$$

Step 4, saturation:

$$S = \begin{cases} 0, & C_{\max} = 0 \\ \frac{\Delta}{C_{\max}}, & \text{otherwise} \end{cases}$$

Step 5, hue:

$$H = \begin{cases} 0^\circ, & \Delta = 0 \\ 60^\circ \times \left(\frac{G' - B'}{\Delta} \right), & C_{\max} = R' \\ 60^\circ \times \left(\frac{B' - R'}{\Delta} + 2 \right), & C_{\max} = G' \\ 60^\circ \times \left(\frac{R' - G'}{\Delta} + 4 \right), & C_{\max} = B' \end{cases}$$

Check: your notes write "mod 360°" after the first case. The modulo is taken on the bracket with 6 (equivalently: compute $60^\circ \times (G' - B')/\Delta$ and add 360° if the answer is negative). Also in the second case you have "B - R"; it is $B' - R'$.

5.4 Worked example 1: generate pink in HSV (your question)

Given $H = 0^\circ$, $S = 0.5$, $V = 1$.

- $V = C_{\max} = 1$, so $R' = 1$ ($R = 255$).
- $S = 1 - C_{\min}/C_{\max} = 0.5$, so $C_{\min} = 0.5$ ($G' = B' = 0.5$, i.e. 127.5).
- $H = 0^\circ$ means red dominant with $G' = B'$.

$$(R, G, B) = (255, 128, 128) \approx \text{pink (light red)}$$

In RGB terms: keep R at 255 and raise G and B equally; pure red (255, 0, 0) has $S = 1$, and lowering S adds white. **Check:** your RGB line for pink appears to read (255, 255, 255) with 0s written beside; that would be white. Pink is (255, 128, 128) (or about (255, 192, 203) for the named colour "pink").

5.5 Worked example 2: RGB (200, 100, 50) to HSV

- $R' = 0.784$, $G' = 0.392$, $B' = 0.196$.
- $C_{\max} = 0.784$ (R'), $C_{\min} = 0.196$, $\Delta = 0.588$.
- $V = 0.784$.
- $S = 0.588 / 0.784 = 0.75$.
- $H = 60^\circ \times ((0.392 - 0.196) / 0.588 \bmod 6) = 60^\circ \times 0.333 = \mathbf{20^\circ}$.

Result: $H = 20^\circ$, $S = 0.75$, $V = 0.784$ (an orange-brown).

5.6 Other colour terms from the syllabus

- **RGB**: additive model for displays. **CMY/CMYK**: subtractive, for printing. **Lab**: perceptually uniform, separates lightness L from colour a , b .
- **YCbCr**: Y = luma, Cb and Cr = blue-difference and red-difference chroma; used in JPEG and video.
- **Pseudocolor**: assigning colours to gray levels (a lookup table) to make small intensity differences visible, e.g. thermal or medical images.

Part 6: Shape features, boundary tracing, Hough transform, MLP

The running problem in this part: **decide whether an object is natural or man-made** using measurable shape features, then feed those features to a classifier (MLP).

6.1 Natural vs man-made objects

Natural objects	Man-made objects
More corners and edges, relatively	Relatively fewer corners and edges
Rarely perfectly symmetric	Highly symmetric, many parallel and perpendicular lines
Irregular mostly, and highly complex	Mostly regular and geometric, with well-delineated boundaries
Fractal in nature	Scale-dependent
Examples: human, tree, mountain	Examples: table, chair, building

Sketch in your notes: a natural outline (person) has many corner points along a curved outline; a man-made outline (rectangle) has only four.

6.2 Shape descriptors

Circularity (also called shape factor or compactness): a dimensionless shape descriptor that quantifies how closely a two-dimensional binary object resembles a perfect circle. It helps classify different kinds of objects and captures roughness of the perimeter and the overall shape form.

$$\text{Circularity} = \frac{4\pi A}{P^2}, \quad A = \text{area}, \quad P = \text{perimeter}$$

- Circularity = 1: perfectly smooth circle. Values near 0: highly elongated, irregular, or jagged shapes.
- Check values: square = $\pi/4 \approx 0.785$; equilateral triangle ≈ 0.605 ; a circle of radius r gives $4\pi(\pi r^2)/(2\pi r)^2 = 1$.

Curvature and radius of curvature:

$$\kappa = \frac{d\theta}{ds}, \quad R = \frac{1}{\kappa}$$

κ measures how sharply the boundary turns per unit path length s ; a straight line has $\kappa = 0$ (R infinite), a sharp corner has a large κ (R small).

Edge density: metric to measure the concentration of edges within a region or across the entire image.

$$\text{Edge density} = \frac{N}{A}, \quad N = \text{total length of edges}, \quad A = \text{total surface area}$$

Check: your page shows the numerator as "N" with a small mark that could be read as a square; the usual definition is N/A . Follow what the slide says.

Convex hull: the smallest boundary that completely encloses a set of points, shapes or pixels, with **no indentations**. **QHull:** Quickhull algorithm to compute convex hulls across multiple dimensions (2D, 3D or higher). Related feature: solidity = object area / convex hull area (close to 1 for man-made compact shapes, lower for natural, indented shapes).

6.3 Binary boundary and contour

- **BW boundary:** a binary boundary, a contour, or a silhouette boundary for the perimeter of a binarized image.
- **Contour:** the continuous, vector-based line that traces the outer limit of a shape or object.
- **Closed path:** returns to the starting point in a continuous manner (like a circular loop).
Open path: one-ended.

Example: a binary image with three separate white objects (a triangle, a circle and a rectangle, made of 1s on a 0 background). In MATLAB:

```
B = \texttt{bwboundaries}(I), \quad \texttt{length}(B) = 3 \quad (\texttt{3 distinct objects})
```

Coordinates example (your page 20/08): a 500×500 binary image, indices 0 to 499 (or 1 to 500 in MATLAB). A filled square of 1s occupies rows and columns 100 to 199 in zero-based indexing, which is 101 to 200 in MATLAB's one-based indexing (100 pixels wide either way). Remember MATLAB starts from 1.

6.4 Moore neighbour tracing algorithm

A **non-parametric technique** used to find and trace the exact continuous contour of a binary object. It moves **clockwise** around the edge pixels of an object to trace its silhouette boundary, based on the **Moore neighbourhood**: the **8 directly surrounding pixels** around a central pixel (a pixel in a grid has 8 neighbours). It extracts a clean **boundary coordinate list**.

Procedure: (1) scan to find the first object pixel (start), (2) note the pixel you entered from (backtrack), (3) examine the 8 neighbours clockwise starting from the backtrack, (4) move to the first object pixel found and update backtrack, (5) stop when you return to the start pixel.

Number-of-boundaries questions (from your page):

Shape	Boundaries	Why
Hollow ring "O"	2	Outer boundary + inner (hole) boundary
Hollow "8"	3	Outer boundary + two hole boundaries
"8" with one loop filled	2	Outer boundary + one remaining hole

6.5 Using Moore's tracing to tell natural from man-made (angle method)

At each step of the traced path, the local direction is:

$$\theta_i = \tan^{-1} \left(\frac{\Delta y}{\Delta x} \right)$$

Then take the derivative of that angle over the path (step distance s), which measures **local curvature**:

$$\frac{d\theta}{ds}$$

- **Natural:** $d\theta$ is constantly changing.
- **Man-made:** $d\theta$ stays almost zero along straight paths, then shows a **sudden spike at sharp corners**.
- When analysing the edge direction θ found through the tracing path, the algorithm looks for a massive spike at $(\theta + 90^\circ)$ (or $\theta + 180^\circ$, $\theta + 270^\circ$) across the entire contour. **If an exact 90° shift keeps occurring, a man-made structure is identified.**

6.6 Hough transform

A technique to isolate specific shapes (**straight lines and circles**) within a **binary (black and white) edge map**. It uses a **global voting procedure in a parameter space** to find geometric shapes even if the shapes are **broken, noisy or disconnected**.

It uses the **polar coordinate system** for a line:

$$\rho = x \cos \theta + y \sin \theta$$

- ρ : perpendicular distance from the origin $(0, 0)$ to the line.
- θ : angle of that perpendicular relative to the horizontal axis.
- Diagram: a line AB; the perpendicular from the origin meets it at (x, y) , with length ρ and angle θ .

Algorithm:

1. **Duality:** a single point (x, y) in image space transforms into a continuous **sinusoidal wave** in Hough parameter space (ρ, θ) .
2. **Accumulator array:** a 2D grid (matrix) of cells whose columns represent values of θ (-90° to 90°) and rows represent values of ρ . This matrix acts as a **voting ballot**.
3. **Voting:** for every single white edge pixel, calculate all possible (ρ, θ) combinations of lines that could pass through that point and add a **+1 vote** to the corresponding accumulator cells.
4. **Peak detection:** if many edge pixels lie along the same straight line, their sinusoidal waves all intersect at **one exact point** in Hough space; the accumulator peak gives that line's (ρ, θ) .

Why it works with noise: isolated noisy pixels scatter votes randomly, while a true line concentrates many votes in a single cell.

6.7 Multi-layer perceptron (MLP) classifier

An MLP classifies objects based on **mathematical input features**. Structure: **Input layer** → **Hidden layers** → **Output layer**. A neuron computes:

$$y = f\left(\sum_i x_i w_i + b\right)$$

x_i = inputs, w_i = weights, b = bias, f = activation function (e.g. ReLU, sigmoid).

Input features for the natural vs man-made MLP:

Feature	Source
Total number of straight lines	Hough transform
Variance of $d\theta$ angular changes	Boundary tracing (Moore's algorithm)
Peak energy at $\theta + 90^\circ$	Boundary orientation
Texture entropy	GLCM

GLCM = Gray-Level Co-occurrence Matrix: counts how often pairs of gray levels occur at a fixed offset. Entropy (randomness of texture):

$$\text{Entropy} = -\sum_i \sum_j p(i, j) \log_2 p(i, j)$$

Natural textures usually have higher entropy than man-made surfaces.

Note (from your page): a **synthetic image** usually has no noise, but a **real (natural) image** will have noise inherently.

6.8 Generative and compression models (mentioned in your notes)

- **Autoencoder:** encoder compresses input to a small code, decoder rebuilds it. It does **compression along with object identification by removing redundant data**.
Denoising autoencoder: trained to reconstruct a clean image from a noisy one.
- **GAN (Generative Adversarial Network):** a generator makes fake images and a discriminator tries to tell fake from real; both improve by competing.
- **Diffusion model:** gradually adds noise to images, then learns to reverse the process to generate or denoise images.
- **Vision Transformer:** splits an image into patches and uses self-attention for global context.

Corrections, formula sheet and practice questions

Corrections to your notes (all in one place)

Where	Your notes	Use this
Intro, selfie example	"Detect noise"	Detect face
Early digital images	Sir Frank Macfarlane Burnet	Bartlane system (Bartholomew and Maynard McFarlane); verify with your slide
NN interpolation example	New (2,2) = old (1,1) = 40	New (2,2) maps to old (1,1) = 10; new (3,3) maps to old (2,2) = 40
Bicubic	"Inverse distance-based weighting"	Cubic convolution over $4 \times 4 = 16$ pixels
Bilinear "why linear"	All neighbour weights equal	Equal only at the exact centre; general weights depend on x and y
Contrast stretching	$\times 2.55$	$\times 255$

Where	Your notes	Use this
Equalization table	Frequencies 8 and 10 at levels 3 and 4	9 and 9 (must match CDF 18/42 and 27/42)
Luminance example	Divide by 3	Standard: $Y = 0.29R + 0.59G + 0.11B$ (224.4); use $\div 3$ only if taught
Laplacian 8-neighbour	Centre 0	Centre 8 (zero-sum)
LoG vs Gaussian	"Because divided by 16"	LoG is zero-sum with negative surround (high-pass); Gaussian is all-positive (low-pass)
Canny	"Hypothesis"	Hysteresis
Hue wheel	Yellow 90°	Yellow 60°
HSV hue formula	$\text{mod } 360; "B - R"$	$\text{mod } 6$ in the first case; $B' - R'$
Pink	RGB (255, 255, 255)	(255, 128, 128)
Gaussian formula	Exponent without σ^2	$e^{\{-(x-\mu)^2/(2\sigma^2)\}}$

Formula sheet (one page to memorise)

Topic	Formula
Levels and storage	$L = 2^b$; size = $M \times N \times b$ bits
Image model	$f(x,y) = i(x,y) \cdot r(x,y)$
Negative	$s = (L - 1) - r$; for 8-bit, $s = 255 - r$
Contrast stretch	$(l - l_{\min}) / (l_{\max} - l_{\min}) \times 255$
Log / exp / power	$g = b \cdot \log(\alpha f + 1)$; $g = b(e^{\alpha f} - 1)$; $g = \alpha f^k$
Equalization	$s_k = \text{round}((L - 1) \cdot \text{CDF}(r_k))$
Convolution	$g(x,y) = \sum \sum K(i,j) f(x - i, y - j)$
NN interpolation	$i_{\text{old}} = i_{\text{new}}[\text{round}(r/sf, c/sf)]$
Bilinear	$P = P11(1-x)(1-y) + P12(1-x)y + P21 \cdot x(1-y) + P22 \cdot xy$

Topic	Formula
Output size (CNN)	$\text{floor}((N + 2p - k)/s) + 1$
Edge strength / direction	$\text{sqrt}(G_x^2 + G_y^2); \theta = \tan^{-1}(G_y/G_x)$
Gaussian kernel 3×3	$(1/16)[1\ 2\ 1; 2\ 4\ 2; 1\ 2\ 1]$
Laplacian 4-neighbour	$[0\ -1\ 0; -1\ 4\ -1; 0\ -1\ 0]$
HSV	$V = C_{\max}; S = \Delta/C_{\max}; H = 60^\circ \times (\text{piecewise})$
Circularity	$4\pi A / P^2$
Curvature	$\kappa = d\theta/ds; R = 1/\kappa$
Edge density	N / A
Hough line	$\rho = x \cos \theta + y \sin \theta$
MLP neuron	$y = f(\sum x_i w_i + b)$
Band ratio (NDVI)	$(\text{NIR} - \text{Red})/(\text{NIR} + \text{Red})$
Distances	$D_E = \text{sqrt}(\Delta x^2 + \Delta y^2); D_4 = \Delta x + \Delta y ; D_8 = \max(\Delta x , \Delta y)$

Short comparisons the exam likes

Pair	Key difference
DIP vs CV	Image to better image, vs image to understanding
Sampling vs quantization	Digitising coordinates (x, y), vs digitising amplitude (intensity)
Spatial vs radiometric resolution	Smallest detail, vs number of bits of intensity
Optical vs digital zoom	Real detail via lens, vs interpolated pixels with no new detail
CCD vs CMOS	Sequential charge readout, low noise, more power; vs per-pixel readout, cheaper, faster
NN vs bilinear vs bicubic	1 pixel copied, blocky; 4 pixels averaged, smooth; 16 pixels cubic, smoothest

Pair	Key difference
Low-pass vs high-pass	Averaging, sum 1, smooths; differencing, sum 0, edges
Prewitt vs Sobel	Equal weights; centre row/column weight 2 (smoother, stronger response)
Edge vs boundary	Any discontinuity; a closed object outline
Multispectral vs hyperspectral	4 to 13 bands; 100 to 300+ narrow bands

Practice questions (answers computed)

- Negative of a pixel with value 87 in an 8-bit image. **168** ($255 - 87$).
- Bilinear value at $x = 0.25$, $y = 0.75$ for $[10\ 20; 30\ 40]$. $10(0.75)(0.25) + 20(0.75)(0.75) + 30(0.25)(0.25) + 40(0.25)(0.75) = 1.875 + 11.25 + 1.875 + 7.5 = \mathbf{22.5}$.
- Contrast-stretch pixel 120 when image min = 60, max = 180. $(120 - 60)/(180 - 60) \times 255 = \mathbf{127.5}$.
- 256×256 image, 5×5 kernel, stride 1, no padding: output size? $256 - 5 + 1 = \mathbf{252 \times 252}$.
- 224×224 image, 3×3 kernel, padding 1, stride 2: $\text{floor}((224 + 2 - 3)/2) + 1 = \text{floor}(111.5) + 1 = \mathbf{112 \times 112}$.
- Sobel horizontal-edge response for 100 above 20. $-(100 \times 4) + (20 \times 4) = \mathbf{-320}$.
- HSV of pure green (0, 255, 0): $C_{\max} = 1$, $\Delta = 1$, so $V = 1$, $S = 1$, $H = 60 \times (0 + 2) = \mathbf{120^\circ}$.
- Hough: point (3, 4), values of ρ at $\theta = 0^\circ, 90^\circ, 45^\circ$. **3, 4, 4.95** (7×0.7071).
- Circularity of a square with side 4. $A = 16$, $P = 16$, so $4\pi \times 16/256 = \pi/4 = \mathbf{0.785}$.
- Storage for a 1024×768 image at 8-bit gray, and at 24-bit colour. 786,432 bytes = **768 KB**; and 2,359,296 bytes = **2.25 MB**.
- Mean filter on $[10\ 20\ 30; 40\ 50\ 60; 70\ 80\ 90]$, centre pixel. Sum = 450, so $450/9 = \mathbf{50}$.
- Which histogram shows the brighter image? The one whose peak lies closer to 255 (Image B, peak at 190, versus Image A, peak at 30).
- How many false colour composites from 3 bands? $3! - 1 = \mathbf{5}$.
- Why is HSV used for colour equalization? Separate RGB equalization changes the balance between channels and distorts colours; equalizing only V keeps hue and saturation.

YouTube resources by syllabus module

These are course and channel names written from memory and **not link-checked**, so search the exact title on YouTube. Tonight, skip long lectures: your notes above are the exam material. If you watch anything, use the short picks in the last row.

Syllabus module	What to search for
Module 1: Introduction, acquisition, sampling, quantization	NPTEL "Digital Image Processing" by Prof. P. K. Biswas (IIT Kharagpur), first lectures on image formation, sampling and quantization
Module 2: Intensity transforms, histograms, equalization	NPTEL Digital Image Processing (same course), lectures on point processing and histogram equalization; MathWorks channel "Image Processing Onramp"
Module 3: Spatial filtering, convolution, noise, median, Gaussian, Laplacian	3Blue1Brown "But what is a convolution?"; Computerphile (Mike Pound) videos on Gaussian blur and convolution
Module 4: Fourier transform, DFT, FFT, frequency filters, wavelets	NPTEL Digital Image Processing frequency-domain lectures; 3Blue1Brown "But what is the Fourier Transform?"
Module 5: Colour models, morphology	NPTEL Digital Image Processing colour and morphology lectures; search "HSV colour model explained" and "dilation erosion opening closing"
Module 6: Edges, corners, SIFT, SURF, ORB	Computerphile Sobel and Canny edge detection; Computerphile "Harris corner" and "SIFT"; Shree Nayar "First Principles of Computer Vision" (Columbia) edge detection and features
Module 7: Segmentation, thresholding, Otsu, watershed, K-means	NPTEL Computer Vision (Prof. Jayanta Mukhopadhyay, IIT Kharagpur) segmentation lectures; search "Otsu thresholding explained"
Module 8: Classical ML for images (KNN, SVM, trees, forests, Naive Bayes)	StatQuest (Josh Starmer) playlists on KNN, SVM, decision trees, random forests, Naive Bayes

Syllabus module	What to search for
Module 9: CNN, transfer learning, YOLO, U-Net	Stanford CS231n and Michigan EECS 498 (Justin Johnson) lectures on CNNs; Ultralytics YOLO channel; search "U-Net explained"
Module 10: Video, optical flow, ViT, DETR, SAM, CLIP	Yannic Kilcher paper walkthroughs of ViT, DETR and CLIP; search "Segment Anything explained"; Computerphile on optical flow
Your Hough, Moore tracing and shape features	Computerphile "Hough transform"; search "Moore neighbour boundary tracing" and "circularity shape factor image processing"

If you only have 40 minutes of video time tonight: 3Blue1Brown on convolution (about 20 min), one Computerphile video on Sobel/Canny, and one histogram equalization walkthrough. Then stop and do the practice questions above without looking.